

LAPORAN AKHIR PROJECT BASED LEARNING (PBL)

Mata Kuliah: Algoritma & Struktur Data (TPL2106)

Tahun Akademik: 2025/2026

Dosen Pengampu: Dr. Shelve Nidya Neyman

SISTEM MANAJEMEN DAN PENELUSURAN RUTE KRL RED LINE BERBASIS DOUBLY LINKED LIST

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL) Mata
Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh Kelompok 14:

Dafi Muzaki Wibisono (J0403251155)

Muhammad Taufan Noviandri (J0403251140)

Aushaf Ghifary Ariyanto (J0403251123)

Program Studi Teknologi Rekayasa Perangkat Lunak Sekolah

Vokasi IPB University

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga proyek akhir Project-Based Learning (PBL) untuk mata kuliah Algoritma & Struktur Data (TPL2106) ini dapat diselesaikan dengan baik. Proyek yang berjudul "Sistem Manajemen dan Penelusuran Rute KRL Red Line Berbasis Doubly Linked List" ini dirancang untuk mensimulasikan serta menyelesaikan permasalahan nyata dalam penelusuran rute transportasi linier dua arah secara efisien.

Kami mengucapkan terima kasih yang sebesar-besarnya kepada Dr. Shelve Nidya Neyman selaku dosen pengampu yang telah memberikan bimbingan, arahan, serta ilmu pengetahuan yang sangat berharga selama perkuliahan. Kami juga berterima kasih kepada seluruh anggota

Kelompok 14 atas dedikasi dan kerja samanya. Laporan ini disusun secara komprehensif untuk mendokumentasikan analisis, rancangan, dan implementasi program yang telah kami bangun.

DAFTAR ISI

1. BAB I. PENDAHULUAN
2. BAB II. ANALISIS DAN PERANCANGAN
3. BAB III. IMPLEMENTASI PROGRAM
4. BAB IV. PENGUJIAN DAN ANALISIS
5. BAB V. KENDALA DAN SOLUSI
6. BAB VI. PEMBAGIAN TUGAS ANGGOTA
7. BAB VII. KESIMPULAN DAN SARAN
8. DAFTAR PUSTAKA
9. LAMPIRAN

BAB I. PENDAHULUAN

1.1 Latar Belakang

Sistem transportasi massal seperti KRL Commuter Line, khususnya Commuter Line Red Line yang melayani rute Bogor menuju Jakarta Kota, memegang peranan krusial dalam mobilitas masyarakat urban. Jalur kereta api ini memiliki karakteristik fisik yang linier namun beroperasi secara dua arah (bidirectional). Pengguna sering kali menghadapi kesulitan dalam memetakan stasiun yang harus dilewati, menentukan jumlah stasiun perantara, serta mengelola data stasiun secara dinamis apabila terjadi perubahan rute operasional atau penambahan stasiun baru. Dalam dunia ilmu komputer, representasi jalur linier dua arah ini sangat ideal jika dimodelkan menggunakan struktur data non-kontigu dinamis. Proyek ini dibangun untuk menyelesaikan permasalahan manajemen stasiun dan penelusuran rute perantara tersebut secara terstruktur. Dengan mengimplementasikan struktur data Doubly Linked List, sistem mampu menghubungkan stasiun satu dengan lainnya menggunakan pointer maju (next) dan pointer mundur (prev). Keunggulan struktur data ini memungkinkan penelusuran rute dilakukan secara fleksibel dari stasiun asal mana pun ke stasiun tujuan mana pun secara interaktif. Selain itu, sistem mengintegrasikan konsep file handling agar seluruh perubahan data stasiun bersifat permanen.

1.2 Tujuan Proyek

1. Membuat aplikasi penelusuran rute perjalanan KRL berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python.

2. Mengimplementasikan konsep struktur data Doubly Linked List secara manual (tanpa library eksternal) untuk memodelkan urutan stasiun kereta api secara logis.
3. Menerapkan metode penelusuran dua arah (forward dan backward traversal) untuk menemukan rute optimal di antara dua stasiun yang dipilih oleh pengguna.
4. Menerapkan mekanisme file handling eksternal untuk menyimpan dan memuat data stasiun kereta api secara persisten, memastikan data tidak hilang saat program ditutup.

1.3 Ruang Lingkup

- **Platform:** Berjalan pada CLI (Console / Terminal Environment).
- **Bahasa Pemrograman:** Menggunakan Python 3 tanpa ketergantungan pada library pihak ketiga.
- **Penyimpanan Data:** Berbasis file teks eksternal (rute_kereta.txt) dengan mekanisme pembacaan baris demi baris.
- **Fitur Utama:** Operasi CRUD stasiun (tambah di awal, setelah stasiun tertentu, dan di akhir rute; hapus stasiun berdasarkan nama/nomor) serta pencarian rute perjalanan dinamis yang menampilkan arah perjalanan, stasiun yang dilewati, dan statistik perpindahan stasiun.

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem Rute KRL Red Line adalah aplikasi konsol interaktif yang mengelola data urutan geografis stasiun kereta api. Secara default, sistem memuat data stasiun dari rute Bogor hingga Jakarta secaraurut. Pengguna disajikan menu utama yang rapi dengan lebar tampilan konsisten (60 karakter). Sistem menyediakan kapabilitas bagi operator untuk memodifikasi rute (menambah atau menghapus stasiun) yang langsung disinkronisasikan ke dalam file teks. Bagi penumpang, sistem menyediakan fitur kalkulasi rute yang mampu menerima masukan berupa nama stasiun maupun nomor indeks numerik untuk kemudahan interaksi, lalu menghasilkan daftar runtut stasiun beserta ringkasan jumlah transit.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi (Deskripsi Detail)	Alasan Pemilihan Struktur Data

Node Class	Menyimpan informasi tekstual nama stasiun (data) serta dua referensi pointer: self.next (menuju stasiun berikutnya) dan self.prev (menuju stasiun sebelumnya).	Berperan sebagai representasi atomik dari sebuah stasiun kereta api terisolasi yang dapat disambungkan secara fleksibel.
Doubly Linked List	Mengelola sekumpulan Node stasiun dengan melacak referensi self.head (awal rute) dan self.tail (akhir rute), serta memelihara variabel ukuran (self.size).	Sifat bidirectional mutlak diperlukan karena KRL Red Line beroperasi dua arah (Bogor ke Jakarta Kota dan sebaliknya). Memudahkan operasi penyisipan atau penghapusan stasiun di tengah rute dengan kompleksitas waktu $O(1)$ setelah node ditemukan, tanpa perlu melakukan pergeseran elemen memori seperti pada Array/List statis.
Python List (Native)	Digunakan sementara pada fungsi dll.to_list() dan dll.show_stations() untuk memetakan referensi memori node menjadi indeks numerik terurut.	Mempermudah pemrosesan antarmuka pengguna, memberikan penomoran otomatis pada stasiun di layar, dan mempercepat proses resolusi input numerik dari user.

2.3 Algoritma yang Digunakan

- **Pencarian Simpul (Node Searching):** Menggunakan algoritma penelusuran linier sekuensial yang dimulai dari head. Pencarian ini mengabaikan sensitivitas huruf kapital dengan mengonversi string masukan dan string target ke metode .lower(). Kompleksitas waktu pencarian ini adalah $O(n)$.
- **Penelusuran Rute Dua Arah (Bidirectional Traversal Routing):** Untuk menemukan lintasan antara stasiun A dan stasiun B, algoritma pertama-tama melakukan penelusuran maju menggunakan pointer '.next' dari simpul asal. Jika simpul tujuan ditemukan dalam jalur maju, lintasan langsung dikembalikan. Jika tidak ditemukan (berarti stasiun tujuan berada di arah sebaliknya), algoritma mereset pencarian dan melakukan penelusuran

mundur menggunakan pointer `.prev` dari simpul asal. Hal ini memastikan rute valid dapat ditemukan secara akurat terlepas dari arah geografis kereta.

- **Mekanisme Resolusi Input:** Sebuah sub-algoritma memeriksa apakah input berupa angka digital (`.isdigit()`). Jika benar, sistem melakukan pemetaan langsung dengan kompleksitas $O(1)$ terhadap array pemetaan sementara. Jika berupa teks, sistem memanggil algoritma pencarian simpul.

2.4 Diagram Alir Program (Flowchart)

2.5 Struktur Program

Aplikasi ini dirancang dengan arsitektur prosedural yang bersih berorientasi objek pada model datanya, terbagi atas komponen logis sebagai berikut:

- **Kelas Komponen Data:** Node dan DoublyLinkedList yang bertindak sebagai mesin pengolah struktur data utama program.
- **Fungsi Antarmuka (Utility):** `garis()`, `garis_tipis()`, `judul()`, `menu()`, `pesan_sukses()`, `pesan_info()`, dan `pesan_error()` yang menjamin konsistensi visual CLI.
- **Fungsi Operasional:** `tambah_stasiun(dll)` dan `hapus_stasiun(dll)` yang mengisolasi input-output pengelolaan data sebelum memodifikasi mesin struktur data.
- **Fungsi Utama (Driver):** `main()` yang mengatur siklus hidup program, inisialisasi awal, pemuatan file teks `rute_kereta.txt`, serta loop menu utama.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Menu utama diimplementasikan dalam fungsi `main()` dengan sebuah loop tak terbatas (`while True`) yang hanya akan berhenti jika pengguna memilih opsi keluar. Visualisasi menu memanfaatkan fungsi dekorator teks terpusat (`.center()`) dengan lebar pembatas 60 karakter untuk menghasilkan estetika antarmuka yang profesional.

3.2 Fitur Create

Penambahan data diakomodasi melalui metode `insert_front(data)` untuk menambahkan stasiun di hulu rute, `insert_end(data)` untuk penambahan di hilir rute, serta `insert_after(target, data)` untuk penyisipan di antara dua stasiun yang sudah ada. Fungsi `tambah_stasiun(dll)` melakukan validasi keunikan string terlebih dahulu agar tidak terjadi duplikasi data stasiun dalam struktur list.

3.3 Fitur Read

Fungsi `dll.show_stations()` melakukan pembacaan sekuensial penuh dari simpul terdepan (`head`) hingga simpul terakhir (`tail`). Setiap stasiun diekstrak namanya, dimasukkan ke dalam List penampung, kemudian dicetak ke layar konsol lengkap dengan nomor urut yang rapi kanan menggunakan pemformatan string tabular.

3.4 Fitur Update

Sistem memperlakukan pembaruan rute secara modular. Modifikasi urutan stasiun dilakukan dengan kombinasi penghapusan simpul lama yang tidak akurat, kemudian memanfaatkan penyisipan dinamis (`insert_after`) untuk meletakkan stasiun pada posisi barunya, menjaga integritas pointer ganda tetap sinkron.

3.5 Fitur Delete

Metode `dll.delete_station(key)` secara presisi memutuskan hubungan simpul yang ditargetkan tanpa merusak rantai list. Algoritma melakukan penanganan kondisi batas secara ketat: pembaharuan `head` jika simpul pertama dihapus, pembaharuan `tail` jika simpul terakhir dihapus, atau menghubungkan langsung pointer `next` simpul sebelumnya dengan pointer `prev` simpul sesudahnya apabila simpul berada di tengah rute.

3.6 Fitur Searching

Implementasi pencarian simpul dijalankan oleh fungsi `dll.find_node(key)`. Penelusuran menggunakan pointer sementara yang terus bergeser (`temp = temp.next`) selama kondisi objek penunjuk tidak bernilai `None`, membandingkan nilai string masukan dengan data internal simpul.

3.7 Fitur Rute Perjalanan

Fitur ini diimplementasikan melalui fungsi `dll.get_route_between(asal, tujuan)`. Fungsi ini mengembalikan sebuah list Python berisi urutan nama stasiun dari stasiun asal hingga stasiun tujuan secara berurutan. Setelah daftar rute diperoleh, fungsi `main()` menggabungkan elemen-elemen tersebut menggunakan separator tanda panah (`->`) untuk merepresentasikan visualisasi perjalanan kereta.

3.8 Penyimpanan Data (File Handling)

Operasi file handling dipisahkan menjadi dua sub-rutin utama yang menjamin persistensi data:

- **Metode `dll.save_file()`:** Membuka file `rute_kereta.txt` dalam mode penulisan ("`w`").

Pointer menelusuri linked list dari awal, menuliskan string nama stasiun diikuti karakter baris baru (\n) ke file, lalu berlanjut hingga tail.

- **Metode `dll.load_file()`:** Memanfaatkan `os.path.exists()` untuk memvalidasi keberadaan file teks. Jika file tersedia, sistem membuka file dalam mode pembacaan ("r"), membersihkan spasi/baris baru dengan metode `.strip()`, lalu menyusun kembali struktur node di memori menggunakan operasi `insert_end()`.

3.9 Validasi Input dan Error Handling

Kasus Masukan (Skenario Error)	Mekanisme Penanganan Sistem (Error Handling)
Input string kosong pada nama stasiun	Fungsi memeriksa nilai panjang string. Jika kosong, fungsi menampilkan pesan_error dan membatalkan operasi penambahan.
Memasukkan stasiun yang sudah terdaftar	<code>dll.find_node()</code> dipanggil sebelum operasi pembuatan simpul baru. Jika mengembalikan simpul (bukan None), sistem menolak masukan.
Stasiun asal dan stasiun tujuan sama	Sistem melakukan validasi komparasi string langsung pada menu penelusuran rute, mencegah perulangan algoritma penelusuran yang sia-sia.
File <code>rute_kereta.txt</code> tidak ditemukan saat startup	Sistem mendeteksi ketiadaan file via modul <code>os</code> , menampilkan pesan informasi informatif, menginisialisasi linked list kosong, dan tidak menyebabkan crash program.
Memasukkan indeks angka di luar jangkauan stasiun aktif	Fungsi <code>dll.resolve_station_input()</code> membatasi input numerik antara 1 hingga <code>dll.size</code> . Di luar jangkauan tersebut, fungsi mengembalikan None.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario Pengujian	Input yang Diberikan	Hasil yang Diharapkan	Status
1	Memuat data rute kereta saat inisialisasi aplikasi	File rute_kereta.txt berisi 24 stasiun aktif	Aplikasi berhasil memuat 24 stasiun ke dalam struktur Doubly Linked List secara berurutan.	Berhasil
2	Menampilkan seluruh daftar stasiun terdaftar (Menu 1)	Opsi menu '1'	Menampilkan tabel rapi berisi daftar 24 stasiun dari stasiun nomor 1 (Bogor) hingga nomor 24 (Jayakarta).	Berhasil
3	Penelusuran rute perjalanan searah rute maju (Menu 2)	Asal: 'Bogor' (atau nomor 1), Tujuan: 'Depok' (atau nomor 6)	Menampilkan lintasan: Bogor -> Cilebut -> Bojonggede -> Cibinong -> Citayam -> Depok. Menghitung jumlah stasiun yang dilewati (6) dan jumlah perpindahan (5).	Berhasil

4	Penelusuran rute perjalanan berlawanan arah rute maju (Menu 2)	Asal: 'Manggarai' (nomor 18), Tujuan: 'Cawang' (nomor 16)	Menampilkan rute mundur: Manggarai -> Tebet -> Cawang. Menampilkan statistik perpindahan secara akurat.	Berhasil
5	Menambahkan stasiun baru di akhir rute (Menu	Nama: 'Jakarta Kota', Opsi Posisi: '3'	Stasiun 'Jakarta Kota' ditambahkan	Berhasil
No	Skenario Pengujian	Input yang Diberikan	Hasil yang Diharapkan	Status
	3 -> 1)		sebagai simpul ekor baru (tail). File diperbarui secara otomatis. Jumlah stasiun aktif bertambah menjadi 25.	
6	Menghapus stasiun berdasarkan indeks nomor (Menu 3 -> 2)	Input: '4' (Stasiun Cibinong), Konfirmasi: 'y'	Stasiun Cibinong berhasil dihapus dari struktur data. Rantai linked list terhubung kembali secara otomatis dari Bojonggede langsung ke Citayam.	Berhasil

4.2 Kelebihan Sistem

1. **Efisiensi Memori Dinamis:** Penggunaan alokasi memori berbasis pointer dinamis memastikan program tidak membuang ruang memori statis secara berlebihan dan mampu tumbuh atau menyusut sesuai dengan perubahan data rute di lapangan.

2. **Penelusuran Rute Fleksibel:** Algoritma penelusuran bidirectional (maju-mundur) memanfaatkan tautan ganda simpul secara optimal, memungkinkannya menemukan rute perjalanan tanpa ketergantungan pada arah urutan data di file teks.
3. **Keamanan Data dan Validasi Berlapis:** Integrasi error handling dan fitur konfirmasi sebelum melakukan penghapusan data mencegah terjadinya kehilangan data yang tidak disengaja oleh operator sistem.

4.4 Keterbatasan Sistem

1. **Keterbatasan Antarmuka:** Aplikasi masih berjalan sepenuhnya pada platform baris perintah (CLI), sehingga kurang intuitif jika dibandingkan dengan aplikasi modern berbasis Graphical User Interface (GUI).
2. **Absensi Sistem Manajemen Database Formal:** Penggunaan flat file (.txt) membatasi performa pembacaan data berskala sangat besar dan belum mendukung mekanisme penguncian data untuk akses bersamaan.
3. **Satu Jalur Tunggal:** Sistem dirancang khusus untuk lintasan linier tunggal (KRL Red Line) dan belum mendukung percabangan rute (interchange stations) yang kompleks seperti rute KRL Blue Line atau rute melingkar.

BAB V. KENDALA DAN SOLUSI

KENDALA	SOLUSI
Implementasi pointer next dan prev pada Doubly Linked List cukup kompleks karena setiap operasi penambahan dan penghapusan node harus menjaga keterhubungan antar node.	Melakukan pengujian setiap operasi (insert_front, insert_end, insert_after, dan delete_station) secara bertahap serta memastikan seluruh pointer diperbarui dengan benar.
Penentuan arah rute perjalanan (maju atau mundur) memerlukan logika tambahan agar sistem dapat menemukan jalur yang benar.	Mengimplementasikan algoritma traversal dua arah (forward dan backward traversal) sehingga sistem dapat mencari rute dari kedua arah.
Risiko kehilangan data ketika pengguna menambah atau menghapus stasiun apabila program ditutup.	Menggunakan mekanisme file handling (save_file() dan load_file()) sehingga seluruh perubahan otomatis disimpan ke file rute_kereta.txt.
Banyak kemungkinan kesalahan input dari pengguna, seperti nama stasiun kosong,	Menambahkan validasi input pada setiap menu serta memberikan pesan kesalahan

stasiun tidak ditemukan, atau nomor indeks tidak valid.	(<i>error handling</i>) yang jelas agar program tidak mengalami crash.
Sinkronisasi antara data yang tersimpan pada file dengan struktur data di memori harus selalu konsisten.	Setiap operasi CRUD langsung memanggil fungsi penyimpanan sehingga isi file selalu sesuai dengan kondisi terbaru pada linked list.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

NAMA	TUGAS
Dafi Muzaki Wibisono (J0403251155)	Mendesain struktur data Node dan Doubly Linked List , mengimplementasikan operasi CRUD (insert, delete), serta mengembangkan mekanisme penyimpanan dan pembacaan data menggunakan file handling.
Muhammad Taufan Noviandri (J0403251140)	Mengembangkan fitur pencarian rute perjalanan (<i>route searching</i>), algoritma traversal dua arah (<i>forward</i> dan <i>backward traversal</i>), validasi input pengguna, serta pengujian fungsi pencarian rute.
Aushaf Ghifary Ariyanto (J0403251123)	Merancang antarmuka Command Line Interface (CLI), menyusun menu program, membuat dokumentasi laporan proyek, melakukan pengujian sistem secara menyeluruh, serta memastikan seluruh komponen program terintegrasi dengan baik.

BAB VII. KESIMPULAN DAN SARAN

7.1 Kesimpulan

- Aplikasi "Sistem Rute KRL Red Line" telah berhasil dibangun menggunakan bahasa Python dengan menerapkan konsep struktur data Doubly Linked List secara fungsional untuk merepresentasikan jalur transportasi kereta api dua arah.
- Operasi manajemen stasiun (CRUD) dan fungsi kalkulasi rute perjalanan terbukti berjalan dengan baik dan stabil berkat adanya mekanisme validasi input serta error handling yang menyeluruh untuk mencegah kegagalan program (crash).

- Mekanisme penyimpanan file handling eksternal yang diimplementasikan menjamin seluruh perubahan data stasiun bersifat persisten, memenuhi standar kebutuhan perangkat lunak manajemen data yang handal.

7.2 Saran Pengembangan

- Mengembangkan antarmuka aplikasi menjadi berbasis grafis (GUI) atau berbasis web interaktif dengan mengintegrasikan peta rute visual agar lebih ramah bagi pengguna awam.
 - Meningkatkan arsitektur struktur data dari Doubly Linked List linier menjadi berbasis Struktur Data Graf (Graph) berbobot agar sistem mampu menangani jaringan kereta api yang lebih kompleks, lengkap dengan stasiun transit, percabangan rute, dan perhitungan algoritma rute terpendek (seperti Dijkstra).
- Mengganti media penyimpanan dari flat file teks menjadi sistem manajemen database relasional (seperti SQLite atau PostgreSQL) untuk menjamin skalabilitas data yang lebih tinggi di masa mendatang.

DAFTAR PUSTAKA

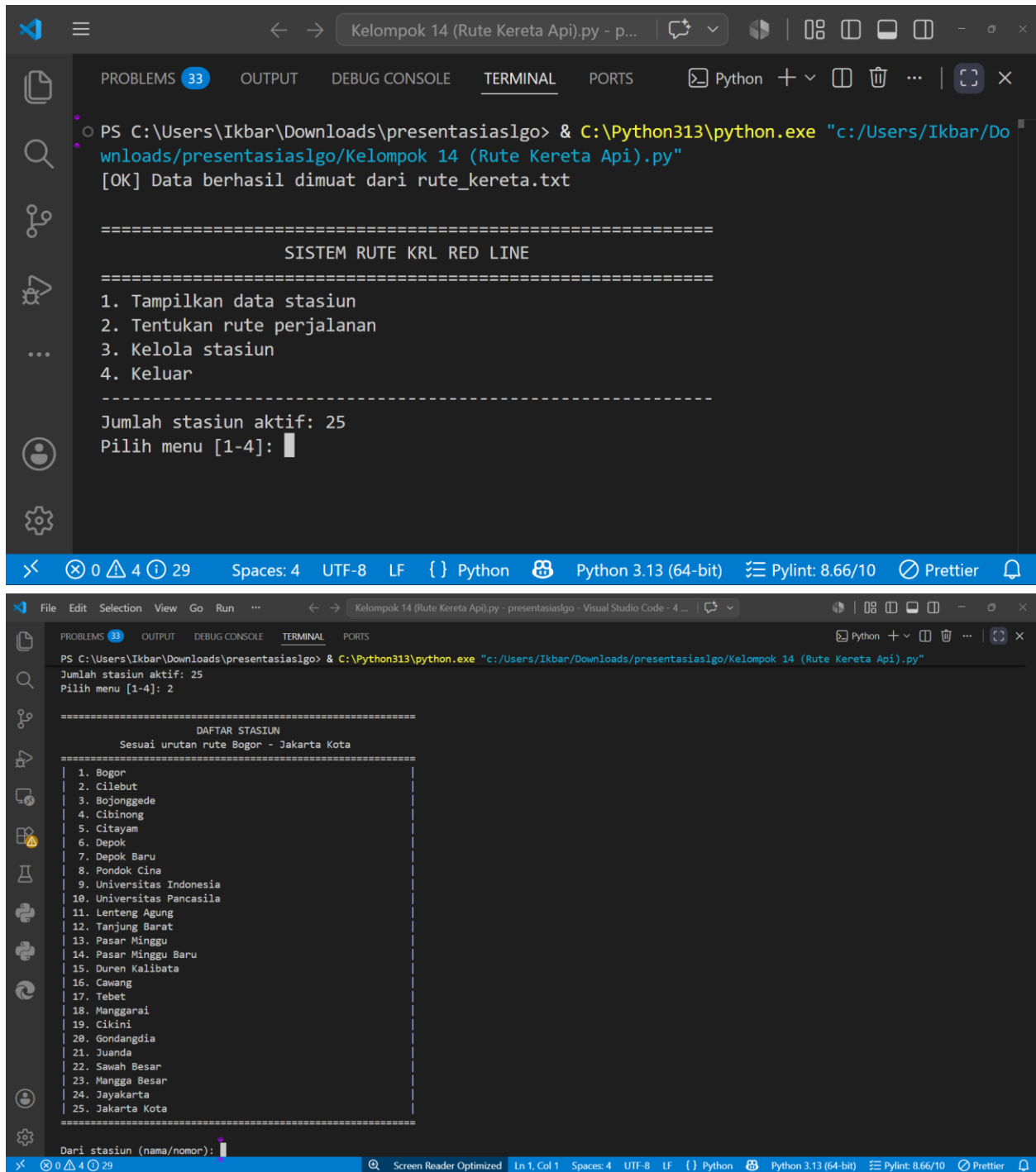
1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
2. M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons, 2013.
3. B. N. Miller and D. L. Ranum, *Problem Solving with Algorithms and Data Structures Using Python*, 2nd ed. Franklin, Beedle & Associates Inc., 2011.

LAMPIRAN

Lampiran A. Link GitHub

https://github.com/taufan9q7/ASD_B_14.git

Lampiran B. Screenshot Program



```
PS C:\Users\Ikbar\Downloads\presentasiaslgo> & C:\Python313\python.exe "c:/Users/Ikbar/Downloads/presentasiaslgo/Kelompok 14 (Rute Kereta Api).py"
[OK] Data berhasil dimuat dari rute_kereta.txt

=====
                SISTEM RUTE KRL RED LINE
=====

1. Tampilkan data stasiun
2. Tentukan rute perjalanan
3. Kelola stasiun
4. Keluar

-----
Jumlah stasiun aktif: 25
Pilih menu [1-4]:
```

```
PS C:\Users\Ikbar\Downloads\presentasiaslgo> & C:\Python313\python.exe "c:/Users/Ikbar/Downloads/presentasiaslgo/Kelompok 14 (Rute Kereta Api).py"
Jumlah stasiun aktif: 25
Pilih menu [1-4]: 2

=====
                DAFTAR STASIUN
        Sesuai urutan rute Bogor - Jakarta Kota
=====
1. Bogor
2. Cilebut
3. Bojonggede
4. Cibinong
5. Citayam
6. Depok
7. Depok Baru
8. Pondok Cina
9. Universitas Indonesia
10. Universitas Pancasila
11. Lenteng Agung
12. Tanjung Barat
13. Pasar Minggu
14. Pasar Minggu Baru
15. Duren Kalibata
16. Cawang
17. Tebet
18. Manggarai
19. Cikini
20. Gondangdia
21. Juanda
22. Sawah Besar
23. Mangga Besar
24. Jayakarta
25. Jakarta Kota
=====

Dari stasiun (nama/nomor):
```

Lampiran C. Struktur Folder Proyek

	taufan9q7 Add files via upload	2e16e91 · 1 hour ago	🕒 4 Commits
	Kelompok 14 (Rute Kereta Api).py	Add files via upload	1 hour ago
	rute_kereta.txt	Add files via upload	1 hour ago

Lampiran D. Commit History GitHub

Commits			
	main		
Commits on Jun 26, 2026			
	Add files via upload		2e16e91
	taufan9q7 authored 1 hour ago		
Commits on May 13, 2026			
	Add files via upload		70d7f9e
	dymzkw authored on May 13		
	update		4794edd
	ArtaxSora committed on May 13		
Commits on Apr 15, 2026			
	Add files via upload		3a6954e
	dymzkw authored on Apr 15		

Lampiran E. Source Code Utama (Python)

```
#===== #
Kelompok 14:
```

```

#
# Dafi Muzaki Wibisono (J0403251155)
# Muhammad Taufan Noviandri (J0403251140)
# Aushaf Ghifary Aryanto (J0403251123)
#
# Rute Kereta Api
#=====

import os

LEBAR_TAMPILAN = 60

def garis():
    print("=" * LEBAR_TAMPILAN)

def garis_tipis():
    print("-" * LEBAR_TAMPILAN)

def judul(teks, subteks=None):
    print()
    garis()
    print(teks.center(LEBAR_TAMPILAN))
    if subteks:
        print(subteks.center(LEBAR_TAMPILAN))
    garis()

def menu(teks, daftar_menu):
    judul(teks)
    for nomor, nama_menu in daftar_menu:
        print(f"{nomor}. {nama_menu}")
        garis_tipis()

def pesan_sukses(teks):
    print(f"[OK] {teks}")

def pesan_info(teks):
    print(f"[INFO] {teks}")

def pesan_error(teks):
    print(f"[!] {teks}")

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

```

```
self.tail = None
self.size = 0
```

```
def isEmpty(self):
    return self.size == 0
```

```
def insert_front(self, data):
    new_node = Node(data)    if
    self.isEmpty():
        self.head = new_node
    self.tail = new_node    else:
        new_node.next = self.head
    self.head.prev = new_node
    self.head = new_node    self.size
    += 1
```

```
def insert_end(self, data):
    new_node = Node(data)    if
    self.isEmpty():
        self.head = new_node
    self.tail = new_node    else:
        self.tail.next = new_node
    new_node.prev = self.tail
    self.tail = new_node    self.size
    += 1
```

```
def insert_after(self, target, data):    target_node =
    self.find_node(target)    if not target_node:    return False
```

```
        if target_node == self.tail:
            self.insert_end(data)    else:
                new_node = Node(data)
            new_node.prev = target_node
            new_node.next = target_node.next
            target_node.next.prev = new_node
            target_node.next = new_node
            self.size += 1    return True
```

```
def delete_station(self, key):
    target_node = self.find_node(key)
    if not target_node:    return None
```

```
        deleted_data = target_node.data
    if self.head == self.tail:
        self.head = None    self.tail =
        None    elif target_node ==
        self.head:    self.head =
        self.head.next    self.head.prev
        = None    elif target_node ==
        self.tail:    self.tail =
```



```

self.tail.prev = self.tail.next =
None else:
    target_node.prev.next = target_node.next
target_node.next.prev = target_node.prev

```

```

    self.size -= 1
return deleted_data

```

```

    def find_node(self, key):
        temp = self.head
        while temp:
            if temp.data.lower() == key.lower():
                return temp
            temp = temp.next
        return None

```

```

    def show_stations(self):
        if self.isEmpty():
            pesan_error("Rute kereta tidak tersedia!")
        return []

```

```

        data = self.to_list()
        judul("DAFTAR STASIUN", "Sesuai urutan rute Bogor - Jakarta Kota")
        for nomor, stasiun in enumerate(data, start=1):
            print(f"| {nomor:>2} | {stasiun:<52} |")
        garis()
        return data

```

```

    def resolve_station_input(self, pilihan, daftar_stasiun):
        if pilihan.isdigit():
            nomor = int(pilihan)
            if 1 <= nomor <= len(daftar_stasiun):
                return daftar_stasiun[nomor - 1]
            return None

```

```

        node = self.find_node(pilihan)
        if node:
            return node.data
        return None

```

```

    def get_route_between(self, asal, tujuan):
        node_asal = self.find_node(asal)
        node_tujuan = self.find_node(tujuan)

```

```

        if not node_asal or not node_tujuan:
            return None
        rute = []
        temp = node_asal
        while temp:
            rute.append(temp.data)
            if temp == node_tujuan:
                return rute
            temp = temp.next
        rute = []
        temp = node_asal
        while temp:

```

```

        rute.append(temp.data)
    if temp == node_tujuan:
    return rute
    temp =
temp.prev

```

```

    return None

```

```

    def to_list(self):
        stasiun =
        temp = self.head
    while temp:
    stasiun.append(temp.data)
    temp = temp.next
    return
    stasiun

```

```

    def clear(self):
self.head = None
self.tail = None
self.size = 0

```

```

    def save_file(self, filename="rute_kereta.txt"):
        try:
            with open(filename,
"w") as f:
                temp = self.head
            while temp:
                f.write(str(temp.data) + "\n")
                temp =
temp.next
            pesan_sukses(f'Data berhasil disimpan ke
{filename}')
        except Exception as e:
            pesan_error(f'Gagal menyimpan data: {e}')

```

```

    def load_file(self, filename="rute_kereta.txt"):
        self.clear()
        if not
os.path.exists(filename):
            pesan_error("File data tidak ditemukan, memulai dengan rute kosong.")
        return
        try:
            with open(filename, "r") as f:
                for line in f:
                    stasiun = line.strip()
                if stasiun:
                    self.insert_end(stasiun)
            pesan_sukses(f'Data berhasil dimuat dari {filename}')
        except Exception as e:
            pesan_error(f'Gagal memuat data: {e}')

```

```

def tambah_stasiun(dll):
    stasiun = input("Masukkan nama stasiun: ").strip()
    if not stasiun:
        pesan_error("Nama stasiun tidak boleh kosong.")
        return
    if
dll.find_node(stasiun):
        pesan_error(f'Stasiun {stasiun} sudah ada dalam rute.')
    return

```

```

menu("POSISI TAMBAH STASIUN", [
    ("1", "Awal rute"),
    ("2", "Setelah stasiun tertentu"),
    ("3", "Akhir rute"),
])
posisi = input("Pilih posisi [1-3]: ").strip()

if posisi == "1":
    dll.insert_front(stasiun)    pesan_sukses(f"Stasiun {stasiun}
berhasil ditambahkan di awal rute.") elif posisi == "2":
    daftar_stasiun = dll.show_stations()    target_input =
input("Tambahkan setelah stasiun (nama/nomor): ").strip()    target =
dll.resolve_station_input(target_input, daftar_stasiun)    if not target:
    pesan_error("Stasiun tujuan penyisipan tidak ditemukan.")
return    dll.insert_after(target, stasiun)    pesan_sukses(f"Stasiun
{stasiun} berhasil ditambahkan setelah {target}.") elif posisi == "3":
    dll.insert_end(stasiun)    pesan_sukses(f"Stasiun {stasiun} berhasil
ditambahkan di akhir rute.") else:
    pesan_error("Pilihan posisi tidak valid.")
return

dll.save_file()

def hapus_stasiun(dll):
    if dll.isEmpty():
        pesan_error("Tidak ada stasiun yang tersedia!")
    return

    daftar_stasiun = dll.show_stations()    hapus_input = input("Hapus
stasiun (nama/nomor): ").strip()    stasiun_hapus =
dll.resolve_station_input(hapus_input, daftar_stasiun)

    if not stasiun_hapus:
        pesan_error("Stasiun yang ingin dihapus tidak ditemukan.")
    return

    konfirmasi = input(f"Yakin hapus stasiun {stasiun_hapus}? (y/n): ").strip().lower()
    if konfirmasi != "y":
        pesan_info("Hapus stasiun dibatalkan.")
    return

    deleted_data = dll.delete_station(stasiun_hapus)    if
deleted_data:    pesan_sukses(f"Stasiun {deleted_data}
berhasil dihapus.")    dll.save_file()

def main():    dll =
DoublyLinkedList()
dll.load_file()

```

```

while True:
    menu("SISTEM RUTE KRL RED LINE", [
        ("1", "Tampilkan data stasiun"),
        ("2", "Tentukan rute perjalanan"),
        ("3", "Kelola stasiun"),
        ("4", "Keluar"),
    ])
    print(f"Jumlah stasiun aktif: {dll.size}")

    pilihan = input("Pilih menu [1-4]: ").strip()

    if pilihan == "1":
        dll.show_stations()

    elif pilihan == "2":
        if dll.isEmpty():
            pesan_error("Rute kereta tidak tersedia!")
            continue

        daftar_stasiun = dll.show_stations()
        asal_input = input("\nDari stasiun (nama/nomor): ").strip()
        tujuan_input = input("Ke stasiun (nama/nomor): ").strip()
        asal = dll.resolve_station_input(asal_input, daftar_stasiun)
        tujuan = dll.resolve_station_input(tujuan_input, daftar_stasiun)

        if not asal_input or not tujuan_input:
            pesan_error("Stasiun asal dan tujuan tidak boleh kosong.")
        elif not asal or not tujuan:
            pesan_error("Stasiun asal atau tujuan tidak ditemukan dalam rute.")
        elif asal.lower() == tujuan.lower():
            pesan_error("Stasiun asal dan tujuan tidak boleh sama.")
        else:
            rute_perjalanan = dll.get_route_between(asal, tujuan)
            if rute_perjalanan:
                judul("RUTE PERJALANAN", f"{asal} menuju {tujuan}")
                pesan_info(f"Arah perjalanan: {asal} menuju {tujuan}")
                print("-> ".join(rute_perjalanan))
                garis_tipis()
                pesan_info(f"Jumlah stasiun yang dilewati: {len(rute_perjalanan)}")
                pesan_info(f"Jumlah perpindahan antarstasiun: {len(rute_perjalanan) - 1}")
            else:
                pesan_error("Rute perjalanan tidak dapat ditemukan.")

    elif pilihan == "3":
        menu("KELOLA STASIUN", [
            ("1", "Tambah stasiun"),
            ("2", "Hapus stasiun"),
            ("3", "Kembali"),
        ])
        pilihan_kelola = input("Pilih menu kelola [1-3]: ").strip()

        if pilihan_kelola == "1":
            tambah_stasiun(dll)
        elif pilihan_kelola == "2":

```

```
hapus_stasiun(dll) elif
pilihan_kelola == "3":
    pesan_info("Kembali ke menu utama.")
else:
    pesan_error("Pilihan kelola tidak valid.")

    elif pilihan == "4":
        judul("TERIMA KASIH", "Program Rute Kereta Api selesai")
break

else:
    pesan_error("Pilihan tidak valid. Silakan pilih menu 1-4.")

if __name__ == "__main__":
    main()
```